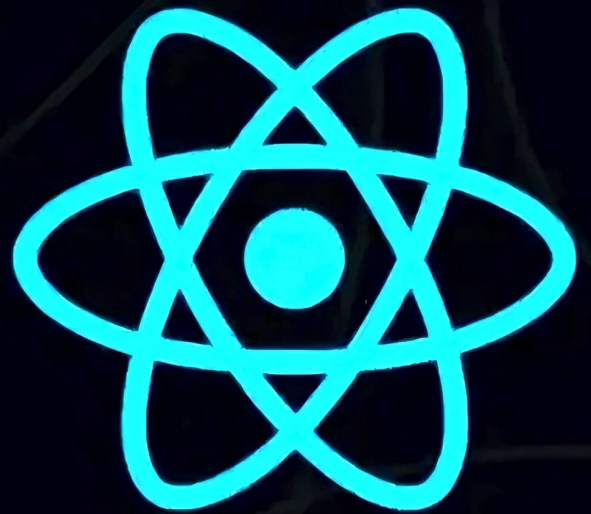


useContext **Hook** **in React**



The **useContext** hook in React allows you to access and share values between components **without passing props** down manually through each level of the component tree.

It's a way to manage state or data that needs to be shared across multiple components.

- **Context**: A way to share data globally for a tree of React components.
- **useContext Hook**: A hook that lets you consume context values directly

Basic Usage

Create Context: Use **React.createContext()** to create a Context object.

Provide Context: Use a **Provider** component to wrap the components that need access to the context.

Consume Context: Use the **useContext** hook to access the context value in any component.



Example

```
import React, { createContext, useContext } from 'react';

// Create a Context
const MyContext = createContext();

// A component that provides the context value
function MyProvider({ children }) {
  const value = 'Hello, World!';
  return <MyContext.Provider value={value}>{children}</MyContext.Provider>;
}

// A component that consumes the context value
function MyComponent() {
  const contextValue = useContext(MyContext);
  return <p>{contextValue}</p>;
}

// Main App component
function App() {
  return (
    <MyProvider>
      <MyComponent />
    </MyProvider>
  );
}

export default App;
```

Explanation

Creating Context: `createContext()` creates a Context object, **MyContext**.

Providing Context: The **MyProvider** component wraps its children with **MyContext.Provider** and provides a value ('Hello, World!') to the context.

Consuming Context: MyComponent uses the **useContext(MyContext)** hook to access the context value ('Hello, World!') and displays it.

This way, **MyComponent** can access the context value directly without needing to receive it as a prop from a parent component.

Problems Solved by useContext:

1. Prop Drilling
2. Global State Management:

Summary :

`createContext()`: Creates a Context object.

Provider: Component that provides a context value to its children.

`useContext()`: Hook to access the context value in a component